1. Definition of JavaScript Syntax

JavaScript syntax is the logic and rule-set that governs how programs are written, structured, and executed by the browser's engine.

Much like human languages have grammar, programming languages have syntax. Without proper syntax, the computer cannot interpret the instructions correctly, leading to 'Syntax Errors'.

Syntax defines the precise placement of keywords, operators, variables, and punctuation (like semicolons and curly braces).

In this guide, we will explore the core pillars of JavaScript syntax in detail.

2. Basic Program Structure

JavaScript consists of multiple 'statements'. A statement is an instruction sent to the computer to perform a specific action.

Example:

```
let x = 10; // Declares variable x
```

```
x = x + 5; // Adds 5 to x
```

Semicolons (;): These are used to separate statements. While JavaScript has Automatic Semicolon Insertion (ASI), manually adding them is a best practice for clean, predictable code.

The execution usually happens from top to bottom, one line at a time.

3. The Art of Commenting

Comments are non-executable parts of your code. They are used to document logic for yourself and other developers.

Single-line Comment: Uses double slashes //.

Multi-line Comment: Wrapped in /* ... */.

Example:

// This is a single line

/* This is a multi-line comment block */

Comments are vital for debugging; you can 'comment out' code to prevent it from running without deleting it.

4. Variables: Declaration & Storage

Variables are symbolic names for values. JavaScript provides three ways to declare them:

```
1. var: The legacy keyword. It has functional scope and can be redeclared.
```

2. let: The modern standard. It has block scope and is used for values that may change.

3. const: Used for constants. Once assigned, the value cannot be modified.

Choosing the right keyword prevents memory leaks and scope-related bugs.

5. Legal Naming Conventions

```
JavaScript Identifiers (names for variables/functions)
must follow strict rules:
```

✔ Must start with a letter (a-z), underscore (_), or dollar sign ($).

✔ Numbers (0-9) are allowed, but NOT at the start of the name.

✔ Case-sensitive: 'myVar' is different from 'myvar'.

```
✔ No Reserved Keywords: You cannot name a variable 'let',
'if', or 'function'.
```

```
Example: let firstName = 'Avni'; // Valid syntax.
```

6. Primitive Data Types

JavaScript handles various types of data implicitly. There are 7 primitive types:

```
1. Number: let price = 99.99;
```

```
2. String: let msg = 'Hello World';
```

```
3. Boolean: let active = true;
```

4. Undefined: A variable with no value.

5. Null: An intentional empty value.

6. Symbol: Unique identifier.

7. BigInt: For very large integers.

7. Arithmetic & Assignment Operators

Operators perform calculations and assignments.

Arithmetic: + (Add), - (Sub), * (Mul), / (Div), % (Mod), ** (Exponent).

```
Assignment: = (Simple), += (Add-Assign), -= (Sub-Assign),
*= (Mul-Assign).
```

Example:

```
let b = 10;
```

```
b *= 2; // b is now 20
```

Postfix/Prefix: i++ and ++i increase values by one.

8. Comparison & Logical Operators

These are used in conditions to control the flow of the program.

```
Equality: == (value) vs === (value + type). Always use
=== for safety.
```

```
Inequality: != and !==.
```

```
Relational: > , < , >= , <=.
```

Logical: && (AND) requires both true, || (OR) requires one true, ! (NOT) flips the value.

```
Example: if(x > 10 && x < 20) { ... }
```

9. Reserved Keywords

Keywords are special words reserved by JavaScript for internal logic.

Control Keywords: if, else, switch, case.

```
Loop Keywords: for, while, do, break, continue.
```

```
Declaration Keywords: let, const, var, function, class,
return.
```

Other: this, new, try, catch, finally.

Since these have built-in meanings, they cannot be used as variable names.

10. Functions: Blocks of Reusable Code

Functions allow you to wrap code and call it when needed. This follows the DRY principle (Don't Repeat Yourself).

Syntax:

```
function greet(name) {
```

return 'Hello ' + name;

}

Execution: greet('Avni');

Functions can accept parameters and return values to the 'calling' part of the script.

11. Conditional Logic: If-Else

Conditional statements decide which code runs based on a boolean condition.

If-Else Ladder:

```
if (score > 90) {
```

```
  grade = 'A';
```

```
} else if (score > 75) {
```

```
  grade = 'B';
```

```
} else {
```

```
  grade = 'C';
```

}

This structure allows for complex decision making inside the browser.

12. Loops: Repeating Logic

Loops handle repetitive tasks efficiently.

For Loop: Used when you know the number of iterations.

While Loop: Used when you iterate until a condition changes.

Do-While Loop: Always runs at least once.

Example for-loop:

```
for (let i = 0; i < 5; i++) {
```

```
  console.log('Counting: ' + i);
```

}

13. Arrays & Indexing

An array is a list-like object used to store multiple values in a single variable.

```
Syntax: let colors = ['red', 'green', 'blue'];
```

Access: colors[0] is 'red'.

Methods: .push() adds items, .pop() removes the last item.

Properties: colors.length tells you how many items are in the list.

14. Objects: Key-Value Pairs

Objects represent real-world entities with properties and methods.

Syntax:

```
let user = {
```

name: 'Avni',

id: 101,

```
  login: function() { console.log('In'); }
```

};

Objects are central to JavaScript (JSON) and are used for API data exchange.

15. Code Style & Whitespace

Whitespace consists of spaces, tabs, and newlines. JavaScript mostly ignores them, but they are crucial for humans.

✔ Indentation: Use 2 or 4 spaces to show nested code.

✔ CamelCase: userFirstName instead of user_first_name.

```
✔ Readable Spacing: let x = a + b; is better than let
x=a+b;.
```

Conclusion: High-quality syntax isn't just about 'working' code; it's about 'readable' code.